Automated E-Commerce Sales Reporting Pipeline: BashOperator and Class-Based Design Implementation

Soichiro Tanabe (24 May 2026)

Table of Contents

1. Executive Summary 2
2. Objectives 2
3. Scope 2
4. Adoption of BashOperator 3
5. Implementation of Class 3
5.1 Advantages 3
5.2 Limitations and Mitigations 3
6. Environment Variable Configuration 4
7. Reference 4

1. Executive Summary

This document presents the technical implementation and design justification for an automated e-commerce sales reporting pipeline built with Apache Airflow and AWS. It builds upon a previous project and introduces two key architectural changes.

First, the TaskFlow API was replaced with the BashOperator, achieving a cleaner separation between orchestration logic and business logic. Second, multiple individual Python files were consolidated into a single class-based module, improving the maintainability and readability of the codebase.

Both decisions were driven by established software engineering principles. The adoption of the BashOperator is supported by the Apache Airflow documentation and industry best practices, which highlight the importance of separating pipeline code from business logic. The class-based design follows the principles of high cohesion and low coupling, ensuring that each component has a single, well-defined responsibility and can be modified or tested independently.

The result is a pipeline that is more maintainable, more resilient, and better structured than its predecessor.

2. Objectives

This project builds upon a previous implementation in which Airflow and AWS were used to automate a data pipeline for e-commerce sales reporting (https://github.com/soichi-berson/Automated_Data_Pipeline_Airflow_AWS). In the original project, the TaskFlow API was adopted alongside multiple individual Python files to handle data analysis.

In this iteration, two key changes were made. First, the TaskFlow API was swapped out for the BashOperator, resulting in a cleaner separation between orchestration logic and business logic. Second, the multiple individual Python files were brought together into a single class-based module, making the codebase easier to maintain and understand.

3. Scope

This document focuses specifically on two architectural changes introduced in this iteration: the adoption of the BashOperator and the implementation of a class-based structure.

For broader context — including the business scenario, system architecture, workflow overview, and technology selection — please refer to the documentation from the previous project, which covers these areas in full detail (https://github.com/soichi-berson/Automated_Data_Pipeline_Airflow_AWS/blob/main/Automated_Data_Pipeline_Airflow_AWS.pdf).

Within this document, each design decision is walked through in terms of how it was implemented and why it was chosen, supported by relevant technical and theoretical references.

4. Adoption of BashOperator

The BashOperator is a built-in Apache Airflow operator that executes a bash command as a task within a DAG. In this project, each task runs a bash command that calls a specific step of the pipeline via the command line, for example:

```
python amazon_daily_class.py --step loading

python amazon_daily_class.py --step clean
```

BashOperator was chosen over the TaskFlow API to achieve a clean separation between orchestration logic and business logic. As noted in the Apache Airflow documentation, separating script logic from pipeline code allows for greater flexibility in structuring pipelines (Apache Airflow, 2024). This is further supported by Raphaelauv (2022), which argues that TaskFlow decorators create high coupling by embedding job logic inside scheduling logic, reducing the ability to maintain and administer DAGs independently.

5. Implementation of Class

5.1 Advantages

The class is designed with high cohesion, where each method has a single, well-defined purpose (loading, cleaning, analysing, and generating the PDF). According to GeeksforGeeks (2025), this brings three key benefits: the code is easier to read and understand, errors are easier to isolate since changes in one method do not affect others, and the overall system becomes more reliable as focused modules are less prone to bugs.

This is further supported by Miller (2008), who argues that high cohesion means a class has a well-defined role within the system, where all responsibilities, data, and methods are closely related to each other. By encapsulating all reporting logic within the `ReportAnalysis` class, each method has a single, well-defined purpose that directly relates to the class name, resulting in code that is easier to read, modify, and test in isolation.

5.2 Limitations and Mitigations

High coupling typically causes three problems: the system becomes harder to understand, components are difficult to modify without breaking others, and modules cannot be tested in isolation (GeeksforGeeks, 2025). However, in this project these disadvantages have minimal impact.

Each method is standalone. For example, `loading()` only loads and `clean_data()` only cleans. They communicate purely through `/tmp/` files rather than calling each other directly. This means each step can be modified or tested independently by simply running:

```
python amazon_daily_class.py --step clean
```

This design achieves low coupling alongside high cohesion, making the pipeline both maintainable and resilient.

6. Environment Variable Configuration

According to the OWASP Foundation (no date), the use of hard-coded credentials is classified as a high severity vulnerability with a very high likelihood of exploit, as it is almost certain that malicious users will gain access through the account in question.

Furthermore, configuration values were stored in environment variables following the Twelve-Factor App methodology (Wiggins, 2012), which states that configuration is everything likely to vary between deployments. According to this methodology, environment variables are preferable because they can be changed between deployments without modifying any code, are less likely to be accidentally committed to a version control repository, and are a language and OS agnostic standard.

Therefore, in this project, all credentials and configuration values are stored in environment variables via a *.env* file rather than hardcoded in the source code

7. Reference

Apache Airflow (2024) *BashOperator — Airflow Documentation*, Apache Airflow. Available at: https://airflow.apache.org/docs/apache-airflow/2.9.0/howto/operator/bash.html (Accessed: 24 May 2026).

GeeksforGeeks (2025) *Coupling and Cohesion - Software Engineering*, GeeksforGeeks, 24 April. Available at: https://www.geeksforgeeks.org/software-engineering/software-engineering-coupling-and-cohesion/ (Accessed: 24 May 2026).

Miller, J. (2008) 'Patterns in Practice: Cohesion and Coupling', *MSDN Magazine*, October. Available at: https://learn.microsoft.com/en-us/archive/msdn-magazine/2008/october/patterns-in-practice-cohesion-and-coupling (Accessed: 24 May 2026).

OWASP Foundation (no date) *Use of Hard-coded Password*, OWASP Foundation. Available at: https://owasp.org/www-community/vulnerabilities/Use_of_hard-coded_password (Accessed: 24 May 2026).

Raphaelauv (2022) *Apache Airflow: 10 rules to make it work (scale)*, Towards Data Science, 13 February. Available at: https://towardsdatascience.com/apache-airflow-in-2022-10-rules-to-make-it-work-b5ed130a51ad/ (Accessed: 24 May 2026).

Wiggins, A. (2012) *The Twelve-Factor App: III. Config*, Heroku. Available at: https://12factor.net/config (Accessed: 24 May 2026).